Practical 6: Perceptron and Multi-Layer Neural Network (MLP) Classification

Name: Aditya Bhende | **Class:** C | **Roll No:** 28 | **Batch:** C2

Aim: Implement the Perceptron Learning Algorithm and a Multi-Layer Neural Network (Keras) for multi-class classification of wheat seeds (`seeds.csv`).

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import (classification_report, accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
tf.random.set_seed(42)
```

2. Load Dataset

Run this cell and **upload your seeds file** (csv or xlsx) from your computer when the file picker appears.

```
from google.colab import files
uploaded = files.upload()
fname = list(uploaded.keys())[0]
print('Loaded file:', fname)

if fname.lower().endswith(('.xlsx', '.xls')):
    df = pd.read_excel(fname)
else:
    df = pd.read_csv(fname)

df = df.dropna(axis=1, how='all')
print('Shape:', df.shape)
df.head()
```

```
Choose Files seeds_new...new (1).csv
seeds_new (1) - seeds_new (1).csv(text/csv) - 9621 bytes, last modified: 10/28/2026 - 100% done
Saving seeds_new (1) - seeds_new (1).csv to seeds_new (1) - seeds_new (1) (1).csv
Loaded file: seeds_new (1) - seeds_new (1) (1).csv
Shape: (210, 8)
```

	Area	Perimeter	Compactness	Length of kernel	Width of kernel	Asymmetry coefficient	Length of kernel groove	Class (1, 2, 3)
0	15.26	14.84	0.8710	5.763	3.312	2.221	5.220	1
1	14.88	14.57	0.8811	5.554	3.333	1.018	4.956	1
2	14.29	14.09	0.9050	5.291	3.337	2.699	4.825	1
3	13.84	13.94	0.8955	5.324	3.379	2.259	4.805	1

Next steps: Generate code with df | New interactive sheet

3. Exploratory Data Analysis (EDA) and Data Cleaning

```
df.info()
print('\nMissing values:\n', df.isnull().sum())
print('\nDuplicate rows:', df.duplicated().sum())
df.describe().T
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   Area                    210 non-null    float64
 1   Perimeter               210 non-null    float64
 2   Compactness             210 non-null    float64
 3   Length of kernel        210 non-null    float64
 4   Width of kernel         210 non-null    float64
 5   Asymmetry coefficient   210 non-null    float64
 6   Length of kernel groove  210 non-null    float64
 7   Class (1, 2, 3)         210 non-null    int64  
dtypes: float64(7), int64(1)
memory usage: 13.3 KB

Missing values:
 Area                      0
Perimeter                 0
Compactness               0
Length of kernel          0
Width of kernel           0
Asymmetry coefficient     0
Length of kernel groove   0
Class (1, 2, 3)           0
dtype: int64

Duplicate rows: 0
```

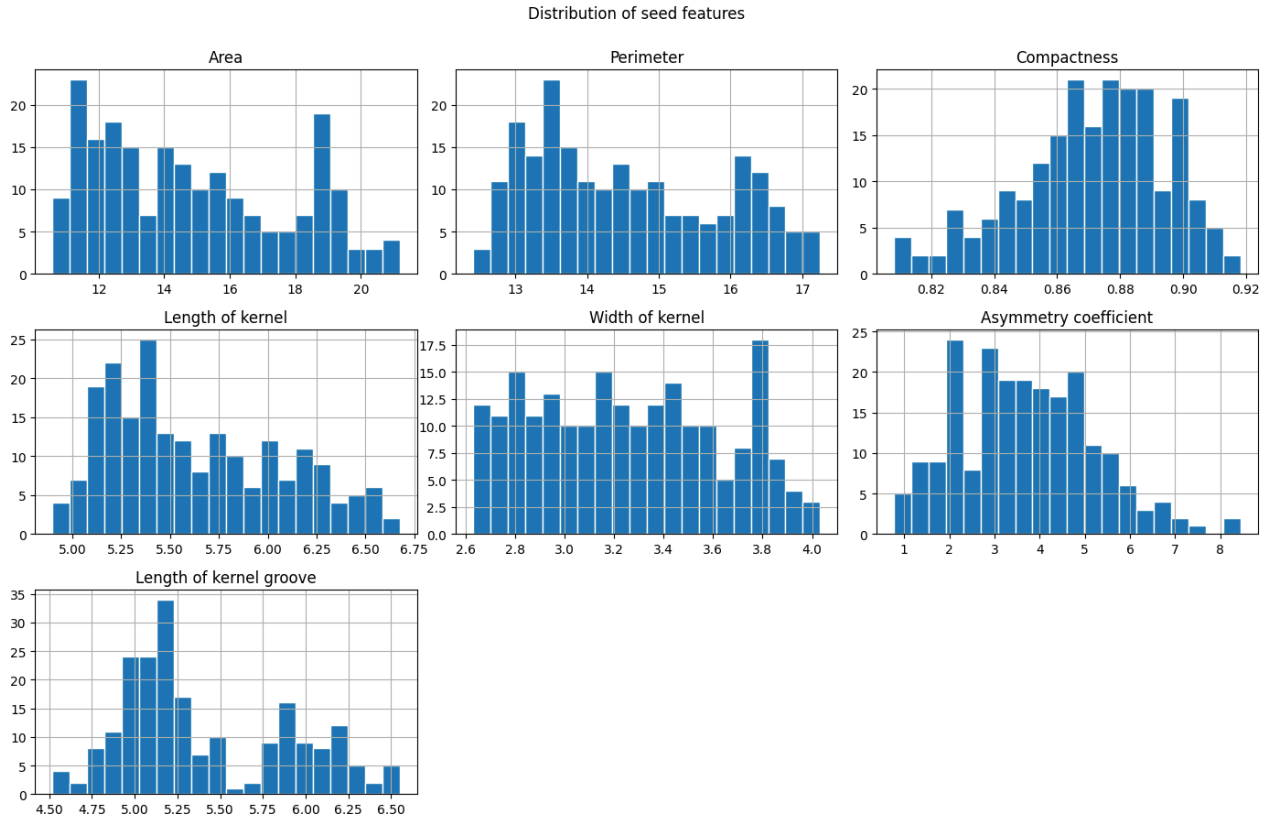
	count	mean	std	min	25%	50%	75%	max
Area	210.0	14.847524	2.909699	10.5900	12.27000	14.35500	17.305000	21.1800
Perimeter	210.0	14.559286	1.305959	12.4100	13.45000	14.32000	15.715000	17.2500
Compactness	210.0	0.870999	0.023629	0.8081	0.85690	0.87345	0.887775	0.9183
Length of kernel	210.0	5.628533	0.443063	4.8990	5.26225	5.52350	5.979750	6.6750
Width of kernel	210.0	3.258605	0.377714	2.6300	2.94400	3.23700	3.561750	4.0330
Asymmetry coefficient	210.0	3.700201	1.503557	0.7651	2.56150	3.59900	4.768750	8.4560
Length of kernel groove	210.0	5.408071	0.491480	4.5190	5.04500	5.22300	5.877000	6.5500
Class (1, 2, 3)	210.0	2.000000	0.818448	1.0000	1.00000	2.00000	3.000000	3.0000

```
target = df.columns[-1]
print('Target column:', target)
print(df[target].value_counts().sort_index())
```

```
Target column: Class (1, 2, 3)
Class (1, 2, 3)
1    70
2    70
3    70
Name: count, dtype: int64
```

Feature Distributions

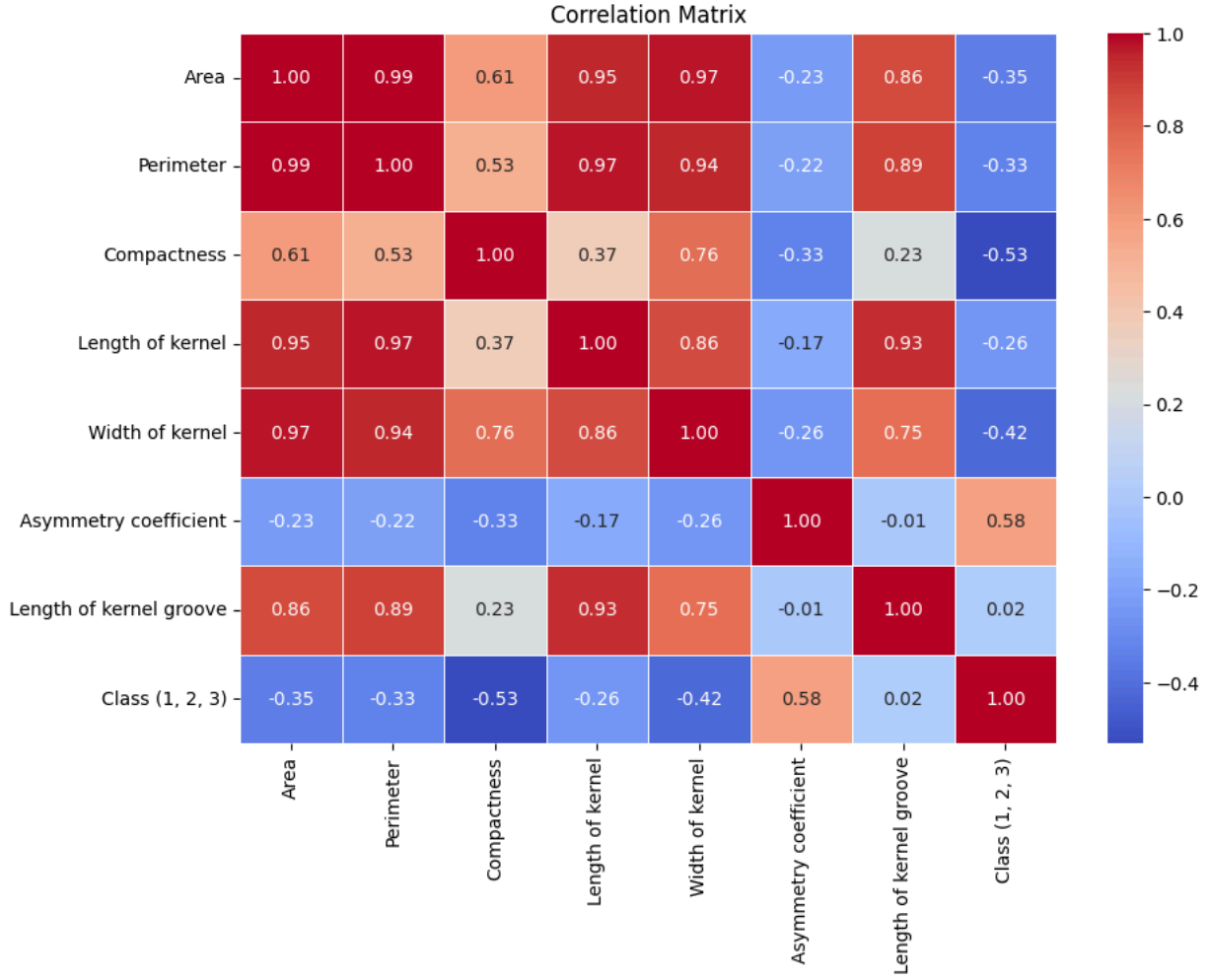
```
df.drop(columns=target).hist(bins=20, figsize=(14, 9), edgecolor='white')
plt.suptitle('Distribution of seed features', y=1.0)
plt.tight_layout()
plt.show()
```



Output 1: Correlation Matrix Heatmap

```
plt.figure(figsize=(10, 7))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()

print(df.corr().round(2))
```



	Area	Perimeter	Compactness	Length of kernel	Width of kernel	Asymmetry coefficient	Length of kernel groove	Class (1, 2, 3)
Area	1.00	0.99	0.61	0.95	0.97	-0.23	0.86	-0.35
Perimeter	0.99	1.00	0.53	0.97	0.94	-0.22	0.89	-0.33
Compactness	0.61	0.53	1.00	0.37	0.76	-0.33	0.23	-0.53
Length of kernel	0.95	0.97	0.37	1.00	0.86	-0.17	0.93	-0.26
Width of kernel	0.97	0.94	0.76	0.86	1.00	-0.26	0.75	-0.42
Asymmetry coefficient	-0.23	-0.22	-0.33	-0.17	-0.26	1.00	-0.01	0.58
Length of kernel groove	0.86	0.89	0.23	0.93	0.75	-0.01	1.00	0.02
Class (1, 2, 3)	-0.35	-0.33	-0.53	-0.26	-0.42	0.58	0.02	1.00

4. Target Label Adjustment, Split and Scaling

Class labels (1, 2, 3) are converted to 0-indexed labels (0, 1, 2) for Keras `sparse_categorical_crossentropy`.

```
X = df.drop(target, axis=1)
y = df[target].astype(int) - 1
print('Classes:', sorted(y.unique()))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print('Train:', X_train.shape, ' Test:', X_test.shape)

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
Classes: [np.int64(0), np.int64(1), np.int64(2)]
Train: (168, 7)  Test: (42, 7)
```

5. Single-Layer Perceptron (Baseline)

```
per = Perceptron(max_iter=1000, random_state=42)
per.fit(X_train, y_train)
per_pred = per.predict(X_test)
print(f'Single-layer Perceptron Test Accuracy: {accuracy_score(y_test, per_pred):.4f}')
```

```
Single-layer Perceptron Test Accuracy: 0.9048
```

6. Multi-Layer Neural Network (MLP)

Output 2: Keras Model Architecture Summary

```
n_features = X_train.shape[1]
n_classes = len(np.unique(y))

model = Sequential([
    Input(shape=(n_features,)),
    Dense(64, activation='relu'),
    Dense(16, activation='relu'),
    Dense(n_classes, activation='softmax')
])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	512
dense_1 (Dense)	(None, 16)	1,040
dense_2 (Dense)	(None, 3)	51

Total params: 1,603 (6.26 KB)
Trainable params: 1,603 (6.26 KB)

```
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
```

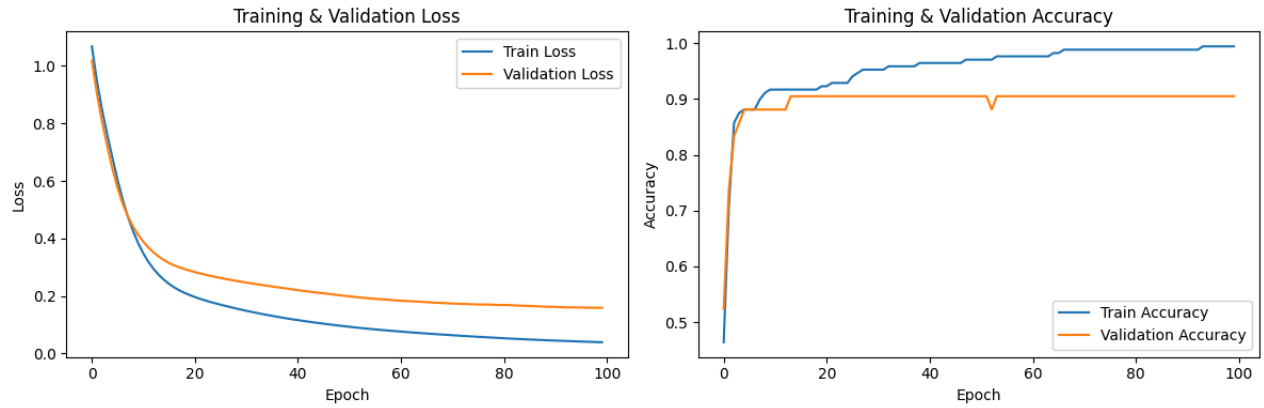
Output 3: Model Training, Loss and Validation Accuracy Curves

```
history = model.fit(X_train, y_train,
                    validation_data=(X_test, y_test),
                    batch_size=32, epochs=100,
                    callbacks=[early_stop])
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0522 - val_accuracy: 0.9048 - val_loss: 0.1685
Epoch 83/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9881 - loss: 0.0513 - val_accuracy: 0.9048 - val_loss: 0.1676
Epoch 84/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0504 - val_accuracy: 0.9048 - val_loss: 0.1665
Epoch 85/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9881 - loss: 0.0496 - val_accuracy: 0.9048 - val_loss: 0.1659
Epoch 86/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9881 - loss: 0.0488 - val_accuracy: 0.9048 - val_loss: 0.1654
Epoch 87/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9881 - loss: 0.0480 - val_accuracy: 0.9048 - val_loss: 0.1645
Epoch 88/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0471 - val_accuracy: 0.9048 - val_loss: 0.1637
Epoch 89/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0465 - val_accuracy: 0.9048 - val_loss: 0.1629
Epoch 90/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0458 - val_accuracy: 0.9048 - val_loss: 0.1624
Epoch 91/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0450 - val_accuracy: 0.9048 - val_loss: 0.1622
Epoch 92/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9881 - loss: 0.0444 - val_accuracy: 0.9048 - val_loss: 0.1612
Epoch 93/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9881 - loss: 0.0437 - val_accuracy: 0.9048 - val_loss: 0.1605
Epoch 94/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9940 - loss: 0.0430 - val_accuracy: 0.9048 - val_loss: 0.1603
Epoch 95/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9940 - loss: 0.0423 - val_accuracy: 0.9048 - val_loss: 0.1603
Epoch 96/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9940 - loss: 0.0417 - val_accuracy: 0.9048 - val_loss: 0.1599
Epoch 97/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9940 - loss: 0.0411 - val_accuracy: 0.9048 - val_loss: 0.1593
Epoch 98/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9940 - loss: 0.0405 - val_accuracy: 0.9048 - val_loss: 0.1592
Epoch 99/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9940 - loss: 0.0398 - val_accuracy: 0.9048 - val_loss: 0.1593
Epoch 100/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9940 - loss: 0.0393 - val_accuracy: 0.9048 - val_loss: 0.1590
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(history.history['loss'], label='Train Loss')
ax[0].plot(history.history['val_loss'], label='Validation Loss')
ax[0].set_xlabel('Epoch'); ax[0].set_ylabel('Loss'); ax[0].set_title('Training & Validation Loss'); ax[0].legend()

ax[1].plot(history.history['accuracy'], label='Train Accuracy')
ax[1].plot(history.history['val_accuracy'], label='Validation Accuracy')
ax[1].set_xlabel('Epoch'); ax[1].set_ylabel('Accuracy'); ax[1].set_title('Training & Validation Accuracy'); ax[1].legend()
plt.tight_layout()
plt.show()
print('Epochs run before early stopping:', len(history.history['loss']))
```




```
Epochs run before early stopping: 100
```

Output 4: Classification Performance and Evaluation Report

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
y_pred = np.argmax(model.predict(X_test), axis=1)

print('Test Evaluation Metrics:')
print('Test Loss     :', test_loss)
print('Test Accuracy :', test_acc)
print('Precision     :', precision_score(y_test, y_pred, average='weighted'))
print('Recall        :', recall_score(y_test, y_pred, average='weighted'))
print('F1 Score      :', f1_score(y_test, y_pred, average='weighted'))
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
Test Evaluation Metrics:
Test Loss      : 0.15898773074150085
Test Accuracy  : 0.9047619104385376
Precision      : 0.9125763125763126
Recall         : 0.9047619047619048
F1 Score       : 0.9057539682539684
```

```
print('Detailed Classification Report:\n')
print(classification_report(y_test, y_pred, target_names=['Class 1', 'Class 2', 'Class 3']))

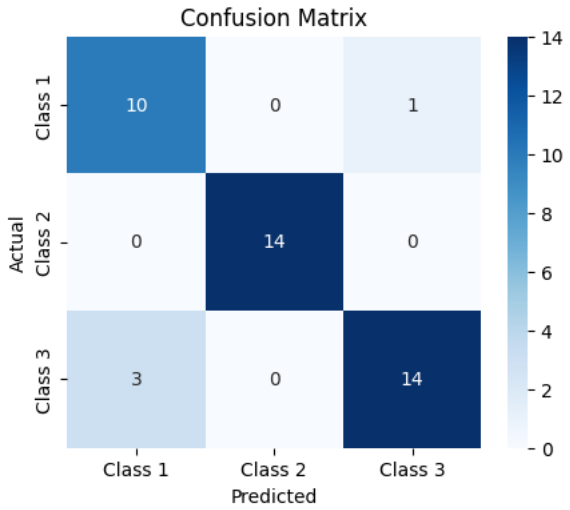
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Class 1', 'Class 2', 'Class 3'], yticklabels=['Class 1', 'Class 2', 'Class 3'])
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title('Confusion Matrix')
plt.show()
```

```
Detailed Classification Report:

              precision    recall  f1-score   support

     Class 1       0.77      0.91      0.83        11
     Class 2       1.00      1.00      1.00        14
     Class 3       0.93      0.82      0.88        17

    accuracy                           0.90        42
   macro avg       0.90      0.91      0.90        42
weighted avg       0.91      0.90      0.91        42
```



Actual vs Predicted

```
results = pd.DataFrame({'Actual': y_test.values + 1, 'Predicted': y_pred + 1})
results['Correct'] = results['Actual'] == results['Predicted']
results.head(15)
```

	Actual	Predicted	Correct
0	1	1	True
1	3	3	True
2	2	2	True
3	3	1	False
4	1	1	True
5	3	3	True
6	1	1	True
7	3	3	True